Statistical Learning: Mental Health Classification Project (Final)

Edgardo Martin Caceres Garcia - Utku Sizanli - Despoina Iapona

2026-03-23

Contents

1 Introduction and Validation Methodology **2**

1.1 **Validation Policy (Addressing Methodology Rigor)** 2

2 Helper Functions **3**

3 Machine Learning Models **4**

3.1 **k-Nearest Neighbors (k-NN)** 4

3.2 **Decision Tree** 7

3.3 **Bagging** 10

3.4 **Random Forest** 13

3.5 **Support Vector Machine (SVM)** 16

3.6 **Neural Network** 18

4 Model Comparison and Conclusion **23**

4.1 Summary of Results 23

4.2 Key Findings 23

4.3 General Observations 24

1 Introduction and Validation Methodology

In this project we will be applying advanced machine learning tools (k-Nearest Neighbors, Decision Trees, Random Forests, and Support Vector Machines) to a dataset of mental health survey responses. The goal is to capture potential non-linear relationships and improve classification performance compared to linear models. We will also perform hyperparameter tuning and model evaluation to identify the best performing model.

1.1 Validation Policy (Addressing Methodology Rigor)

To ensure a robust evaluation and avoid data leakage, we will utilize a strict three-way data split:

- **Training Set (60%)**: Used exclusively for model fitting and applying SMOTE to balance the classes.
- **Validation Set (20%)**: Used for model selection and hyperparameter tuning, and for optimizing decision thresholds via Cost-Sensitive Learning.
- **Test Set (20%)**: Used only for the final evaluation of the selected model, providing an unbiased estimate of true model performance on unseen data.

```r
library(caret)
library(MASS)
library(dplyr)
library(e1071)
library(pROC)
library(ggplot2)
library(tidyr)
library(patchwork)
library(DMwR)
library(kernlab)
library(randomForest)
library(tree)
library(class)
library(tensorflow)
library(purrr)
library(forcats)
library(keras3)

mental     <- read.csv("mental_health.csv")
target_col <- "Has_Mental_Health_Issue"

mental$Has_Mental_Health_Issue <- factor(
  mental$Has_Mental_Health_Issue,
  levels = c("0", "1"), labels = c("No", "Yes")
)
mental[] <- lapply(mental, function(x) if (is.character(x)) as.factor(x) else x)

set.seed(42)
idx_train    <- createDataPartition(mental$Has_Mental_Health_Issue, p = 0.60, list = FALSE)
mental_train <- mental[idx_train, ]
mental_tmp   <- mental[-idx_train, ]
idx_val      <- createDataPartition(mental_tmp$Has_Mental_Health_Issue, p = 0.50, list = FALSE)
mental_val   <- mental_tmp[idx_val, ]
mental_test  <- mental_tmp[-idx_val, ]
```

With this rigorous data splitting strategy, we ensure that our model evaluation is unbiased and that the performance metrics we report are reflective of how the model will perform on truly unseen data.

2 Helper Functions

Four helper functions are defined here and reused across all models throughout this analysis.

`calc_metrics_bal` computes classification metrics (Accuracy, Precision, Sensitivity, Specificity, Balanced Accuracy, F1) for a given probability threshold. Balanced Accuracy is used as the primary metric because the dataset has a class imbalance (majority = Yes).

`pick_best_thr_bal` sweeps over a grid of thresholds and selects the one that maximises Balanced Accuracy, with ties broken by Sensitivity then Specificity. Applied to validation set probabilities for each model.

`plot_threshold_effect_one` visualises how metrics change as the decision threshold varies, with a vertical line marking the selected best threshold.

`prepare_data` applies the full data preparation pipeline: subsets to feature columns, applies SMOTE to the training set only (`perc.over = 600, perc.under = 100`), and standardises numeric predictors using statistics from the SMOTE'd training set.

```r
calc_metrics_bal <- function(y_true, prob_yes, thr) {
  pred <- factor(ifelse(prob_yes >= thr, "Yes", "No"), levels = c("No", "Yes"))
  cm   <- caret::confusionMatrix(pred, y_true, positive = "Yes")
  acc  <- as.numeric(cm$overall["Accuracy"])
  sens <- as.numeric(cm$byClass["Sensitivity"])
  spec <- as.numeric(cm$byClass["Specificity"])
  bal  <- 0.5 * (sens + spec)
  prec <- as.numeric(cm$byClass["Pos Pred Value"])
  f1   <- if (is.na(prec) || is.na(sens) || (prec + sens) == 0) NA_real_
          else 2 * prec * sens / (prec + sens)
  data.frame(threshold = thr, Accuracy = acc, Precision = prec,
             Sensitivity = sens, Specificity = spec, BalancedAcc = bal, F1 = f1)
}

pick_best_thr_bal <- function(y_true, prob_yes) {
  tbl <- dplyr::bind_rows(lapply(thr_grid, function(t) calc_metrics_bal(y_true, prob_yes, t))) |>
    dplyr::arrange(dplyr::desc(BalancedAcc), dplyr::desc(Sensitivity), dplyr::desc(Specificity))
  list(best = tbl[1, ], all = tbl)
}

plot_threshold_effect_one <- function(tbl_all, model_name, best_thr = NULL) {
  tbl_long <- tbl_all %>%
    dplyr::select(threshold, Accuracy, Precision, Sensitivity, Specificity) %>%
    tidyr::pivot_longer(cols = -threshold, names_to = "Metric", values_to = "Value")
  p <- ggplot(tbl_long, aes(x = threshold, y = Value, color = Metric)) +
    geom_line(linewidth = 1) + geom_point(size = 1.4) +
    labs(title = model_name, x = "Probability Threshold", y = "Metric Value") +
    coord_cartesian(ylim = c(0, 1)) +
    theme_minimal(base_size = 12) +
    theme(plot.title       = element_text(face = "bold", size = 14),
          legend.position  = "bottom",
          panel.grid.minor = element_blank()) +
    scale_color_viridis_d(end = 0.9)
```

```r
  if (!is.null(best_thr) && is.finite(best_thr))
    p <- p + geom_vline(xintercept = best_thr, linetype = "solid",
                        color = "black", linewidth = 0.6)
  p
}

scale_apply <- function(df, num_cols, mu, sd) {
  df[, num_cols] <- sweep(sweep(df[, num_cols], 2, mu, "-"), 2, sd, "/")
  df
}

prepare_data <- function(train, val, test, feature_cols, scale = TRUE) {
  keep     <- c("Has_Mental_Health_Issue", feature_cols)
  tr       <- train[, keep]; va <- val[, keep]; te <- test[, keep]
  tr_smote <- SMOTE(Has_Mental_Health_Issue ~ ., data = tr,
                    perc.over = 600, perc.under = 100)
  tr_smote$Has_Mental_Health_Issue <- factor(tr_smote$Has_Mental_Health_Issue,
                                             levels = c("No", "Yes"))
  if (scale) {
    num_cols <- setdiff(names(tr_smote)[sapply(tr_smote, is.numeric)],
                        "Has_Mental_Health_Issue")
    mu <- colMeans(tr_smote[, num_cols])
    sd <- apply(tr_smote[, num_cols], 2, sd); sd[sd == 0] <- 1
    list(train = scale_apply(tr_smote, num_cols, mu, sd),
         val   = scale_apply(va, num_cols, mu, sd),
         test  = scale_apply(te, num_cols, mu, sd))
  } else {
    list(train = tr_smote, val = va, test = te)
  }
}

thr_grid <- seq(0.05, 0.95, by = 0.01)
```

3 Machine Learning Models

For all models, we evaluate performance using **Balanced Accuracy** and **ROC-AUC** rather than simple accuracy. In a dataset where 92% of patients have mental health issues, a model predicting "Yes" for every observation would achieve 92% accuracy while being clinically useless. Balanced Accuracy accounts for class imbalance by averaging recall across classes, while ROC-AUC evaluates discrimination across all thresholds.

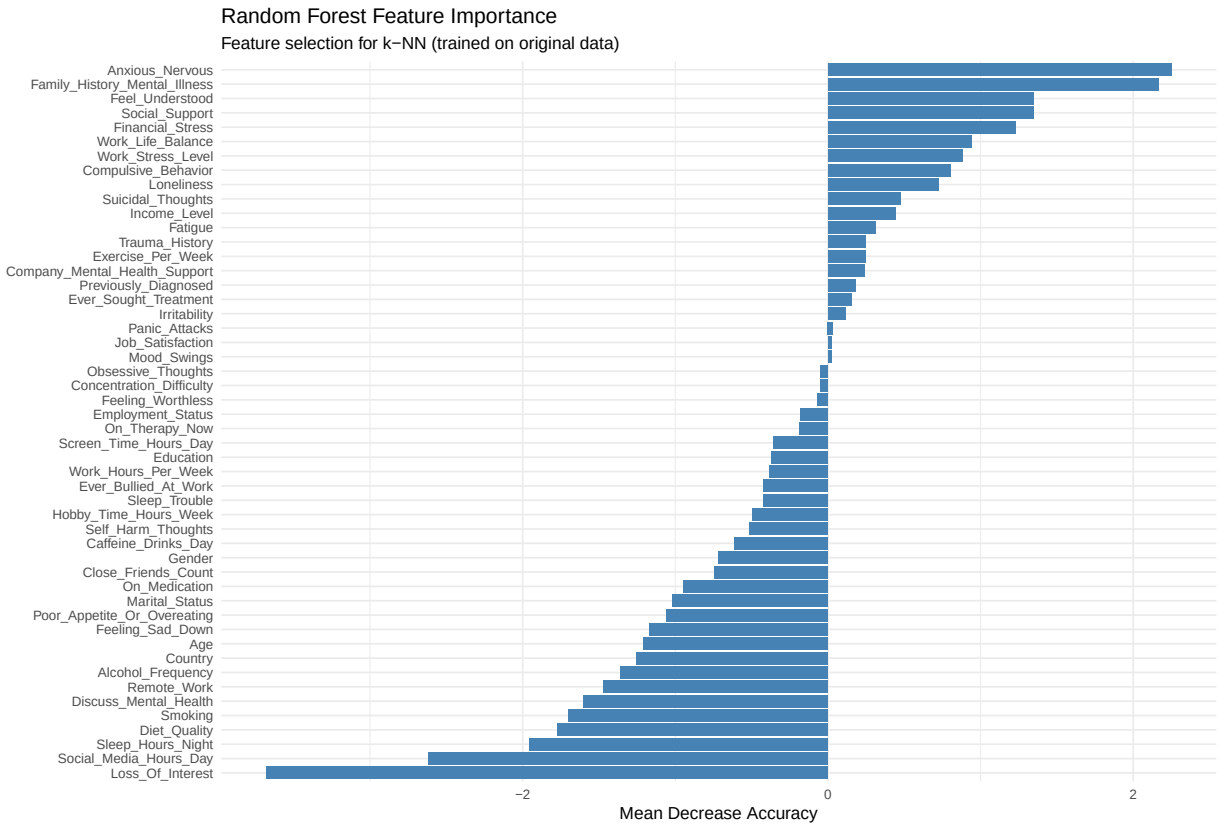
3.1 k-Nearest Neighbors (k-NN)

The k-Nearest Neighbors algorithm classifies observations based on the majority vote of its k closest neighbors in the predictor space. Because it relies entirely on Euclidean distance, the quality and scale of the predictor set is critical.

3.1.1 Feature Selection

k-NN has no mechanism to discount irrelevant features – every feature contributes to the distance calculation equally. Random Forest importance (Mean Decrease Accuracy) was used to rank predictors. Feature selection

was performed on the original training data before SMOTE, ensuring importance scores reflect the true signal rather than synthetic observations. Features with importance below the mean of positive importance values were excluded, as were binary and categorical variables (for which Euclidean distance is not meaningful).




```
## Final kNN features: 8
```

The final feature set consists of **8 Likert-scale predictors**: `Anxious_Nervous`, `Feel_Understood`, `Social_Support`, `Financial_Stress`, `Work_Life_Balance`, `Work_Stress_Level`, `Compulsive_Behavior`, and `Loneliness`.

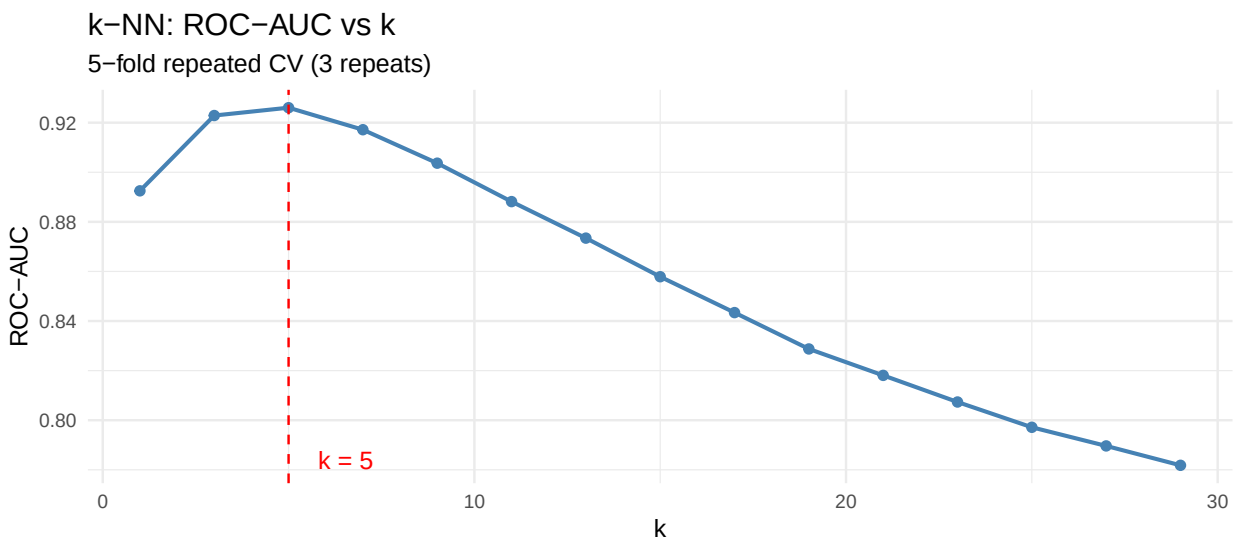
3.1.2 Data Preparation

SMOTE was applied via `prepare_data` to balance the training set, followed by standardisation of all numeric predictors. Scaling parameters were computed from the SMOTE'd training set and applied consistently to validation and test sets to prevent data leakage.

The SMOTE'd training set contains 6123 observations across 8 features.

3.1.3 Hyperparameter Tuning: Selecting k

The number of neighbours k was selected via 5-fold repeated cross-validation (3 repeats) using ROC-AUC as the tuning metric. Only odd values were considered to avoid ties in the majority vote.

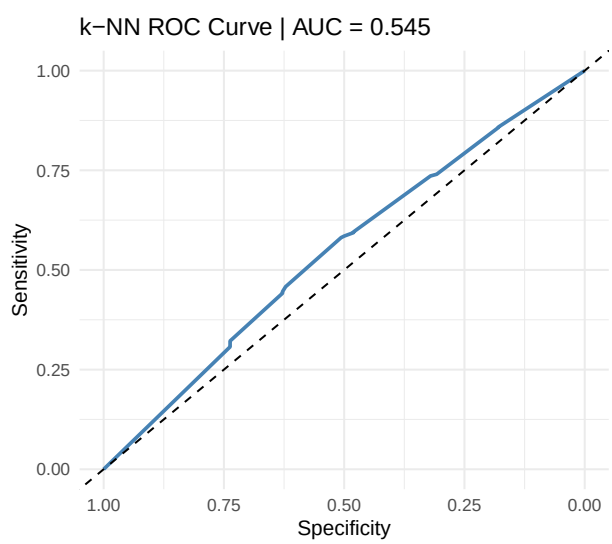


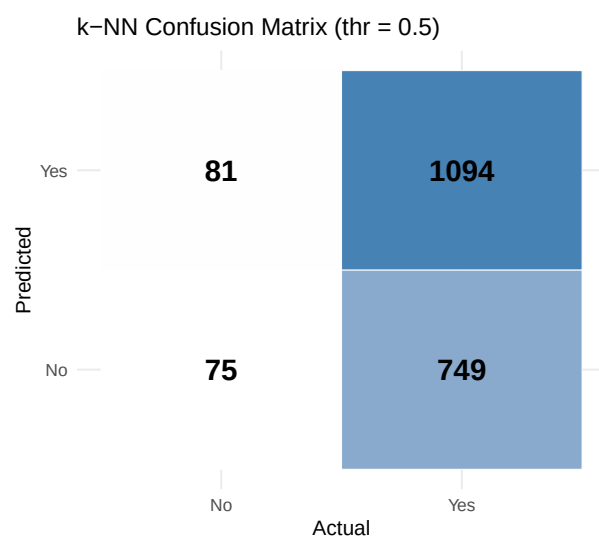
The plot shows a clear peak at $k = 5$, after which ROC-AUC decreases monotonically. The CV ROC of 0.926 is evaluated on SMOTE-balanced folds and does not reflect performance on the true class distribution – the test set evaluation below provides the reliable estimate.

3.1.4 Threshold Selection

k-NN probability estimates are inherently coarse: with k neighbours, only $k + 1$ distinct probability values are possible. With $k = 5$, only 6 unique probabilities exist, making threshold optimisation unreliable. The default threshold of 0.50 was therefore retained.

3.1.5 Test Set Evaluation





The test AUC of 0.545 is only marginally above random guessing (0.5), confirming that k-NN does not generalise well to this dataset. The large gap between CV ROC (0.926) and test AUC (0.545) is a direct consequence of SMOTE inflating performance on balanced training folds.

3.1.6 Decision Boundary

To visualise the model's decision structure, a 2D decision boundary was constructed using the two most important features. All remaining features were fixed at their training set mean, and jitter was applied to account for the discrete nature of Likert-scale values.

The boundary is highly irregular and fragmented – characteristic of k-NN with small k – and does not align well with the observed class separation. The considerable overlap between Yes and No classes confirms the poor discriminative performance.

3.1.7 Why k-NN is Unsuitable for This Dataset

1. Ordinal/Likert predictors. All 8 selected features are Likert scales (1-10). k-NN computes Euclidean distances between them, assuming equal intervals – an assumption that does not hold for ordinal data.

2. SMOTE distortion. SMOTE generates synthetic observations by interpolating between existing ones, producing non-integer values (e.g. `Anxious_Nervous = 3.7`) that distort the distance space.

3. Curse of dimensionality. With 8 features, observations become increasingly equidistant, making the neighbourhood selection essentially arbitrary.

3.2 Decision Tree

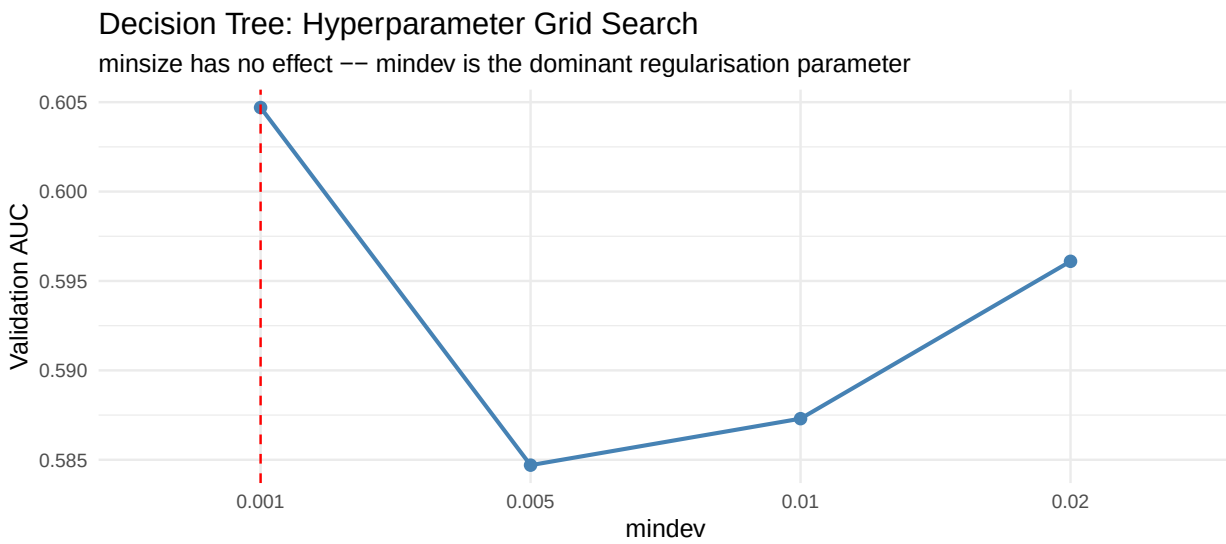
A single decision tree provides an interpretable, hierarchical view of the mental health predictors. Every decision can be traced back to a specific feature and threshold.

3.2.1 Data Preparation

All features were passed through `prepare_data` with `scale = FALSE`. Scaling is intentionally omitted for tree-based models – decision trees split on feature thresholds, so the absolute scale of predictors does not affect the splitting criterion. The same unscaled SMOTE'd data is reused across Decision Tree, Bagging, and Random Forest.

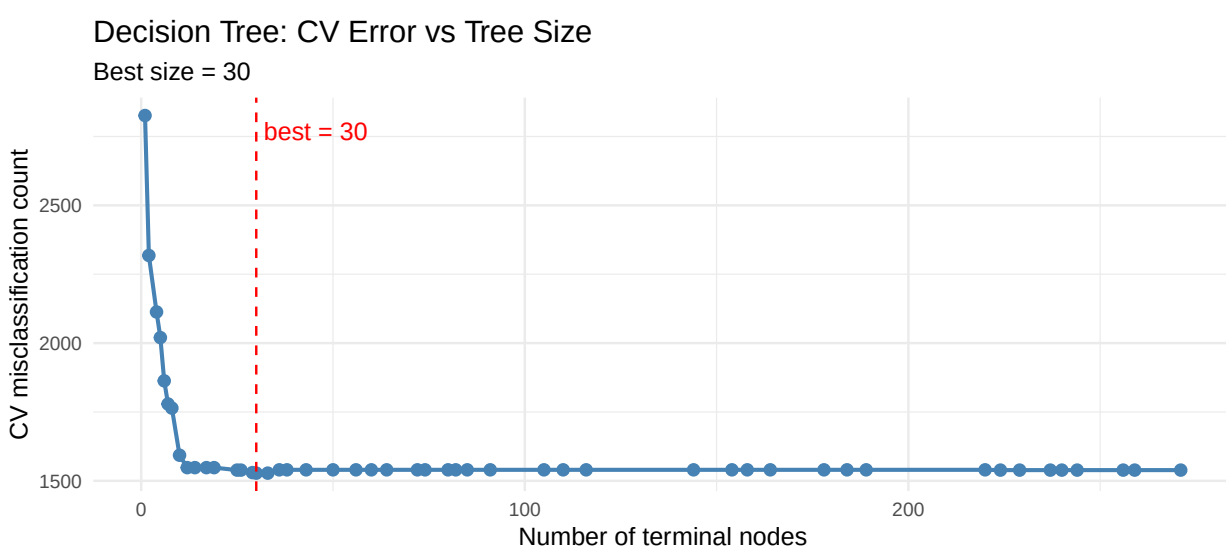
3.2.2 Hyperparameter Tuning

Two hyperparameters control tree complexity before pruning: `mindev` (minimum deviance reduction required for a split) and `minsize` (minimum observations in a node). A grid search was performed over `mindev` in {0.001, 0.005, 0.01, 0.02} and `minsize` in {5, 10, 20, 30}, evaluating each combination by validation ROC-AUC after 5-fold CV pruning.

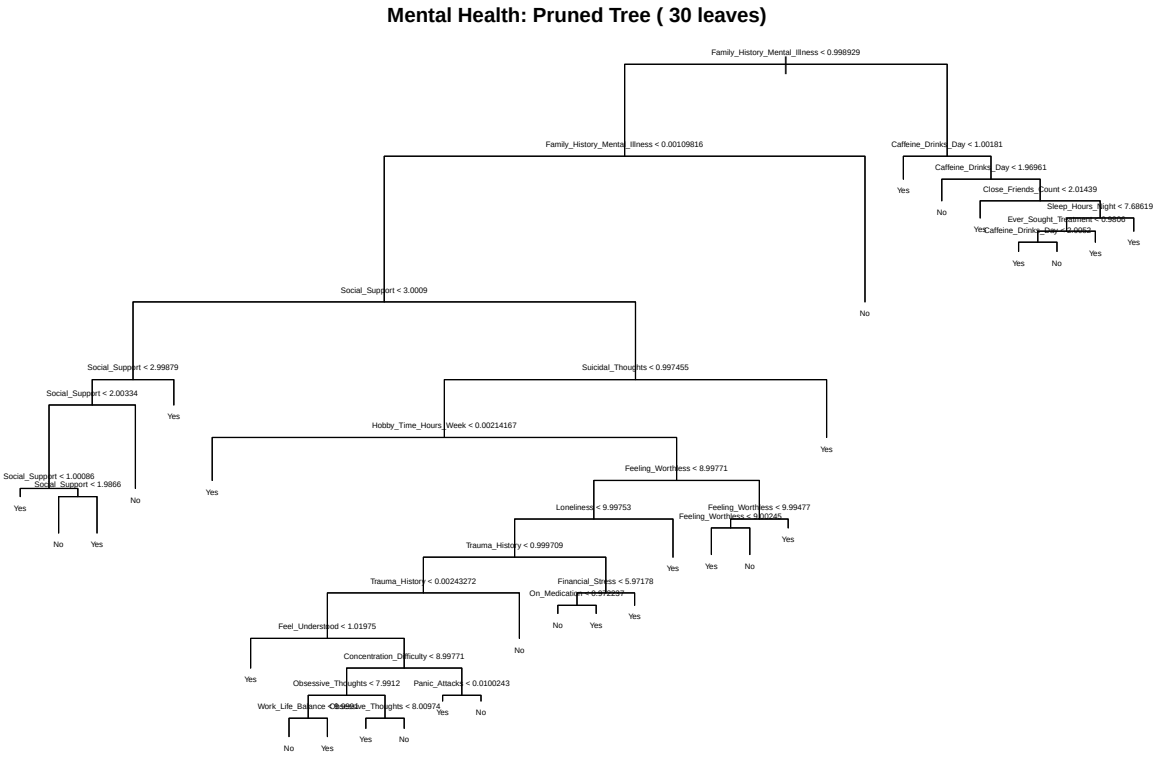


The grid search reveals that `minsize` has no meaningful effect – with over 6,000 training observations after SMOTE, the constraint rarely binds. The best combination is `mindev = 0.001`, `minsize = 5`, yielding validation AUC of 0.6047.

3.2.3 Pruning via Cross-Validation

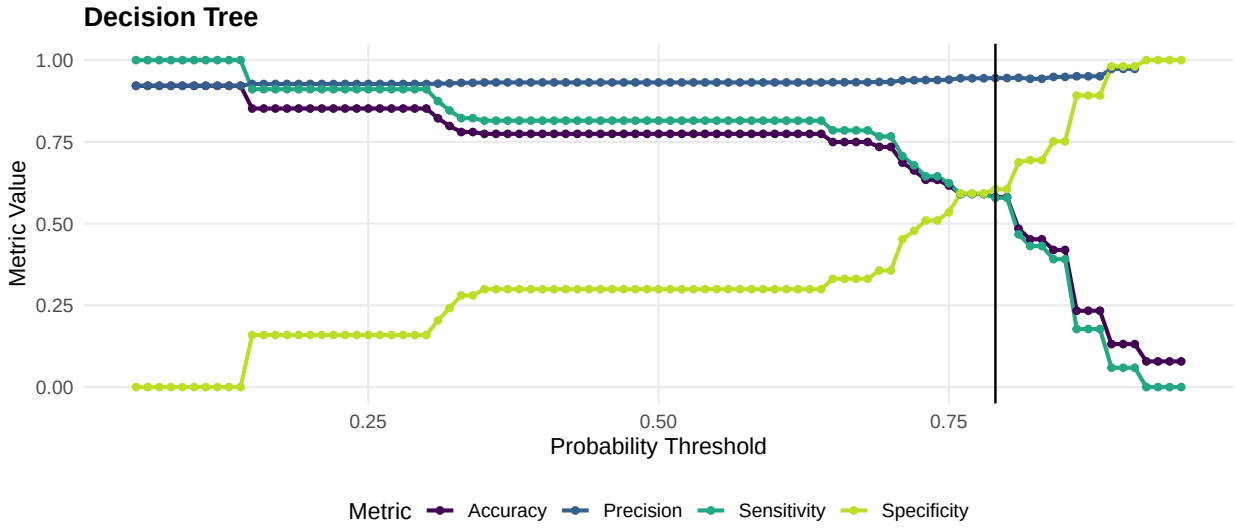


The full tree with 271 terminal nodes is pruned to 30 leaves. CV error drops sharply up to ~40 nodes then plateaus – additional splits no longer reduce misclassification.



`Social_Support` appears as the root split – the single most informative predictor. Subsequent splits introduce `Family_History_Mental_Illness`, `Suicidal_Thoughts`, `Feeling_Worthless`, `Work_Stress_Level`, and others, capturing both environmental and psychological dimensions.

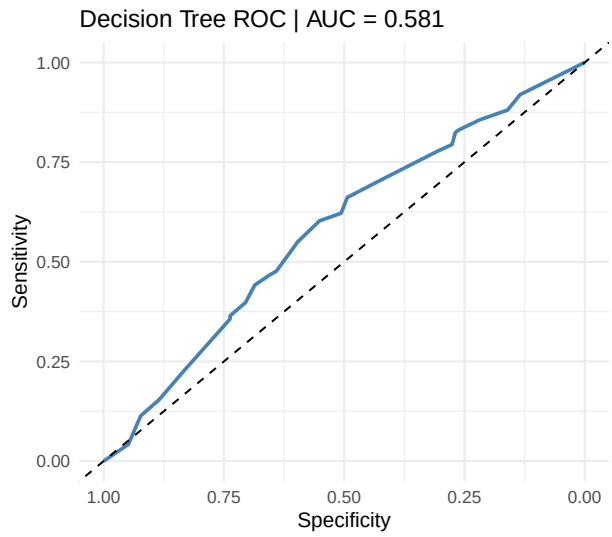
3.2.4 Threshold Selection

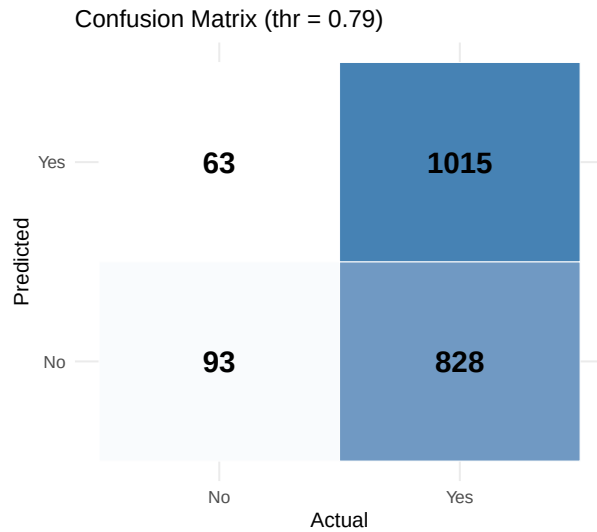


The optimal threshold of 0.79 was selected by maximising Balanced Accuracy on the validation set. This high value reflects that SMOTE inflates "Yes" probabilities in leaf nodes – a high threshold compensates by

requiring stronger evidence before predicting "Yes". The discrete jumps in the plot are consistent with the finite number of unique probability values from a 30-leaf tree.

3.2.5 Test Set Evaluation





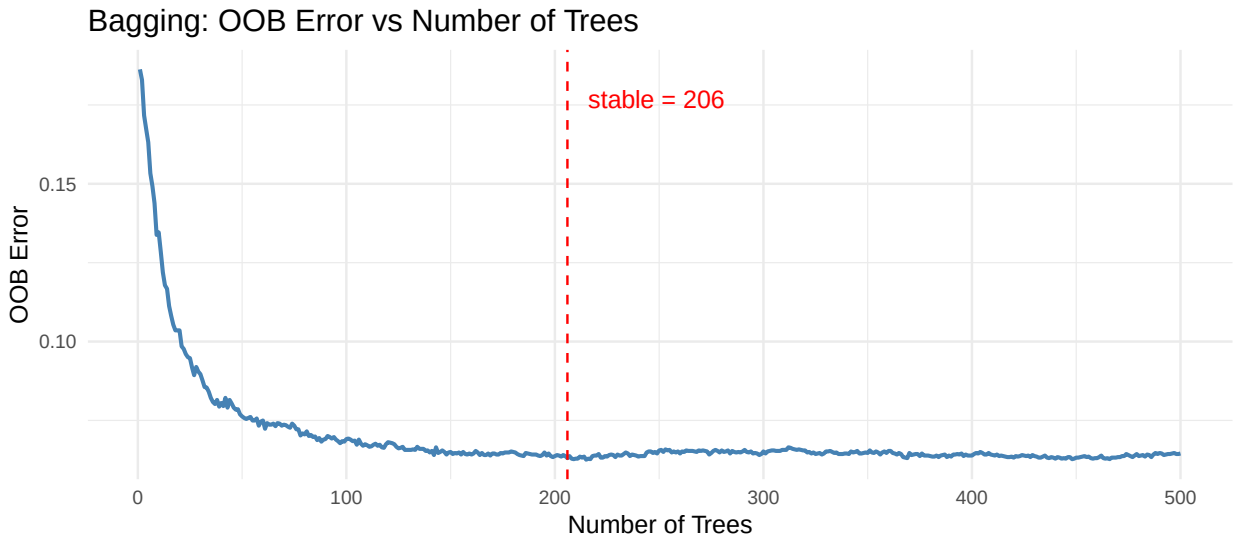
The decision tree achieves a test AUC of 0.581, modestly above k-NN. The ROC curve shows lift above the diagonal, confirming the model captures real signal. The 30-leaf tree provides clinically credible and communicable rules – however, a single tree suffers from high variance, motivating the transition to Bagging and Random Forest.

3.3 Bagging

Bagging (Bootstrap Aggregating) extends the single decision tree by training many trees on bootstrap samples and averaging their predictions, directly addressing the high variance of a single tree. Unlike Random Forest, Bagging uses all predictors at every split (`mtry = p`), so the trees are highly correlated.

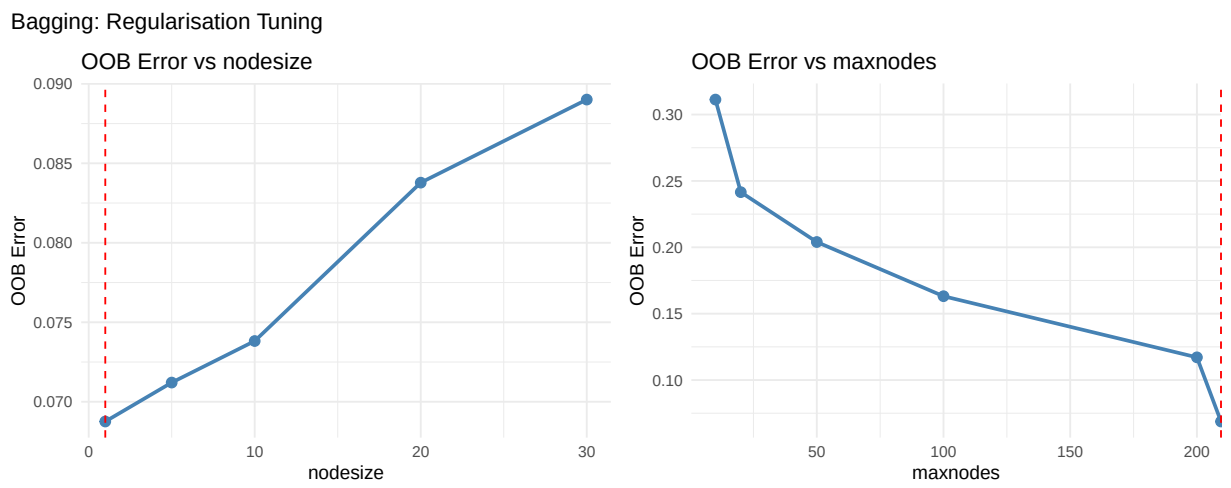
3.3.1 Number of Trees

`ntree` was selected based on OOB error stabilisation – more trees never hurt, they simply stabilise performance.



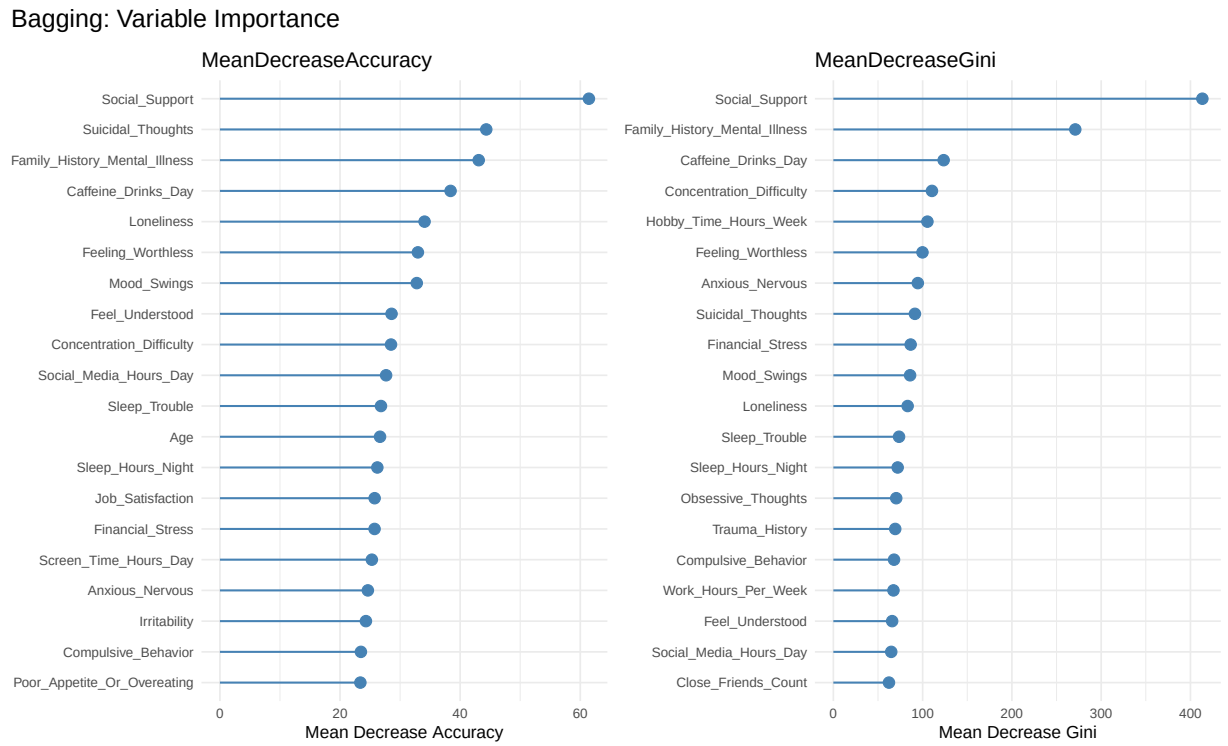
OOB error stabilises at tree 206. All subsequent tuning uses this value.

3.3.2 Regularisation: nodesize and maxnodes



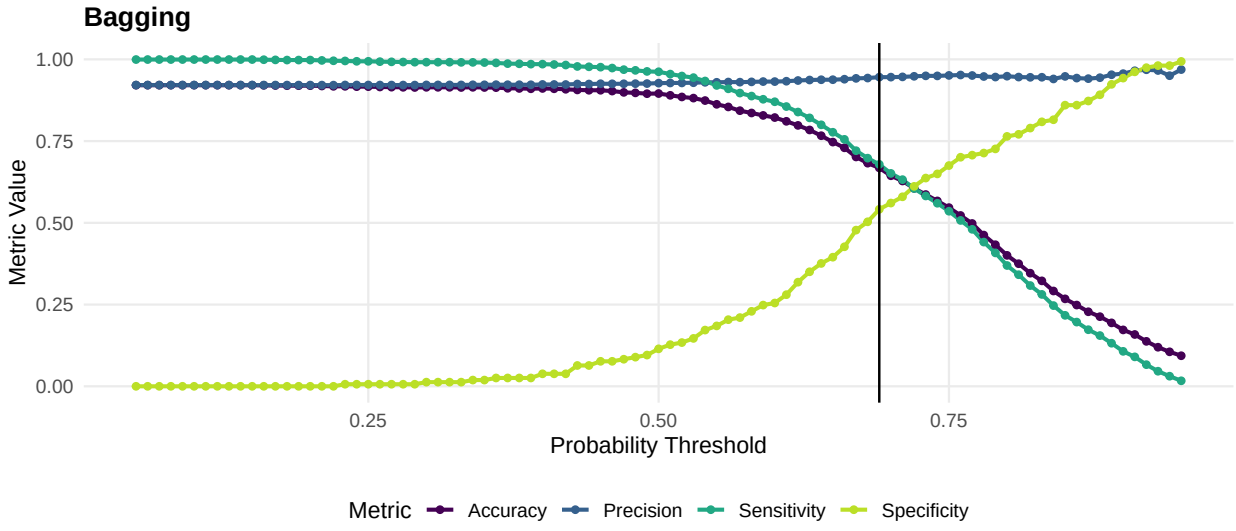
OOB error increases monotonically with `nodesize` – best is `nodesize = 1` (fully grown trees). Similarly, unrestricted `maxnodes` achieves the lowest error. For bagging, variance reduction from averaging compensates for the high variance of individual deep trees, making regularisation counterproductive.

3.3.3 Final Model



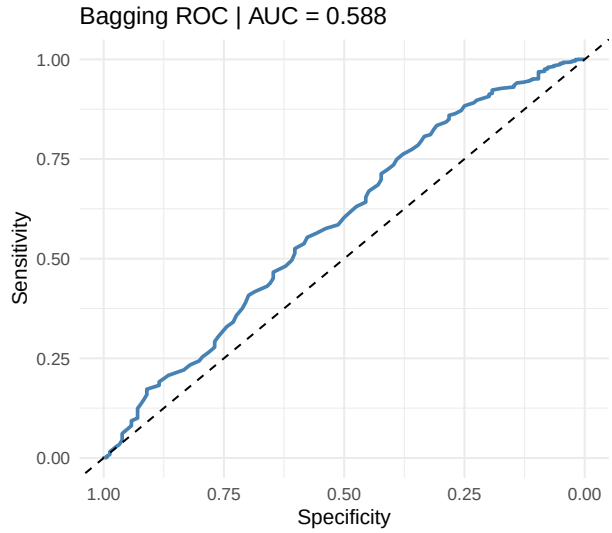
`Social_Support` is the most important predictor by both MeanDecreaseAccuracy and MeanDecreaseGini, consistent with its role as the root split in the pruned tree. `Suicidal_Thoughts` and `Family_History_Mental_Illness` also rank highly.

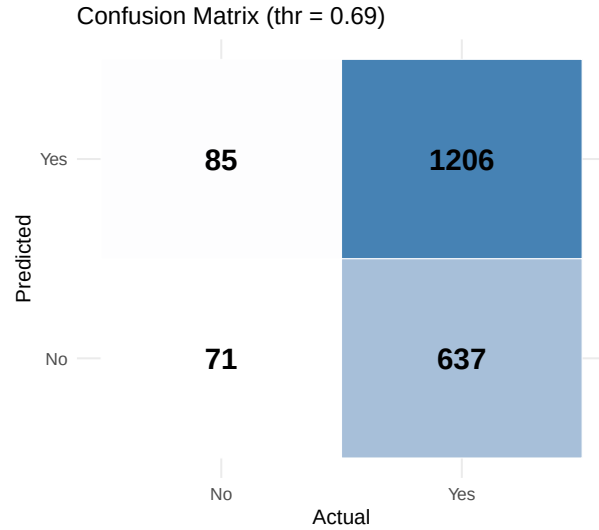
3.3.4 Threshold Selection



The smooth threshold curves reflect bagging's well-averaged probability estimates. The optimal threshold of 0.69 is higher than the default 0.50, reflecting upward probability bias introduced by SMOTE.

3.3.5 Test Set Evaluation



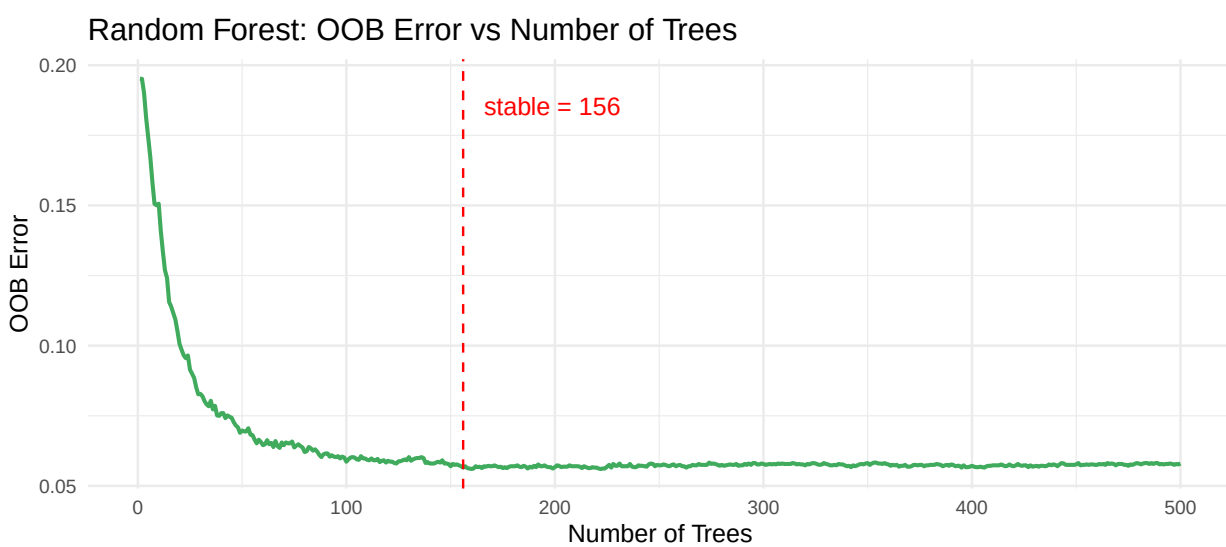


Bagging achieves test AUC of 0.588, a meaningful improvement over the single decision tree (0.581). The improvement is real but limited – because all predictors are considered at every split, individual trees are highly correlated. Random Forest addresses this directly.

3.4 Random Forest

Random Forest introduces an additional source of randomness: only a random subset of `mtry` predictors is evaluated at each split. This decorrelates the individual trees, producing a more diverse ensemble with greater variance reduction.

3.4.1 Number of Trees

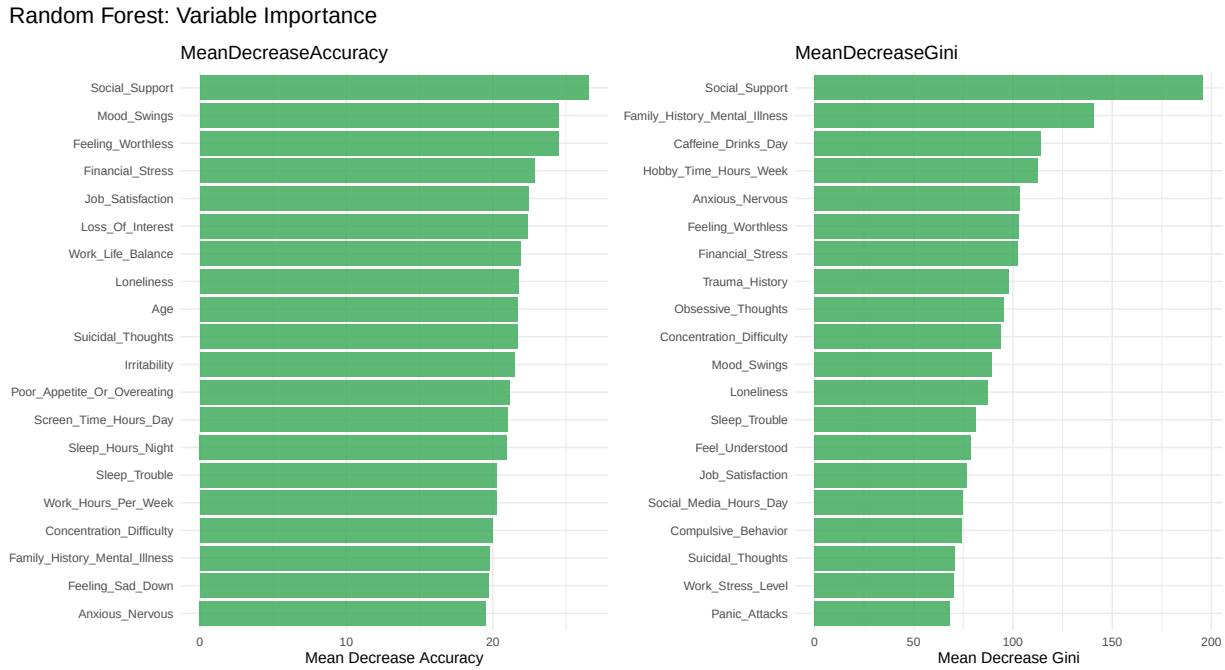


OOB error stabilises at tree 156 – faster than Bagging because decorrelated trees converge more efficiently.

3.4.2 Hyperparameter Tuning: mtry, nodesize, maxnodes

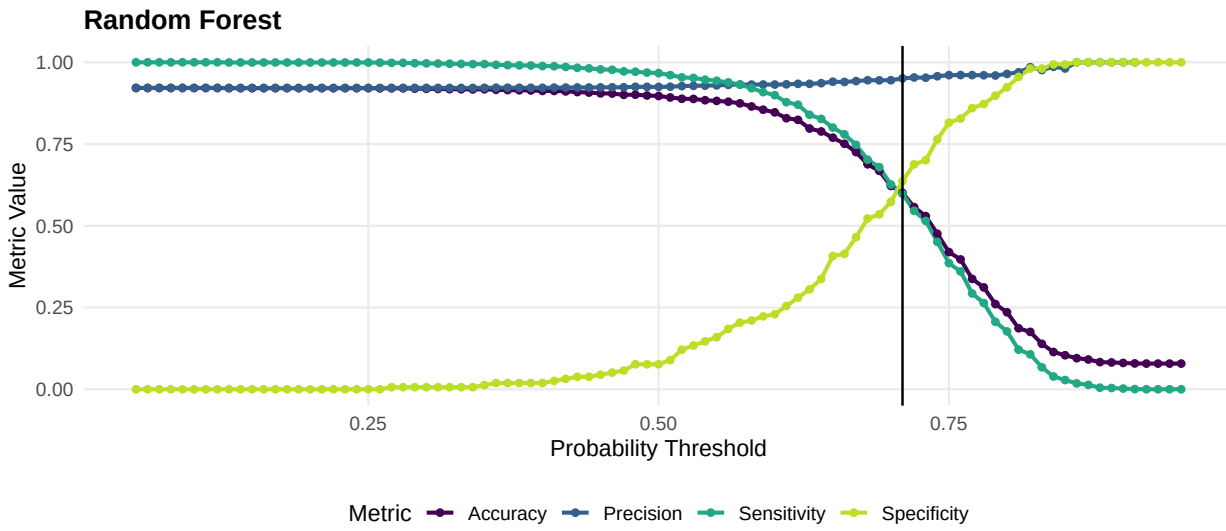
The optimal `mtry = 8` is below the standard default of $\lfloor\sqrt{p}\rfloor = 7$. Dominant predictors like `Social_Support` cause correlation when always available – restricting to 8 candidates forces the ensemble to explore a broader range of features. As with Bagging, `nodesize = 1` and unrestricted `maxnodes` are optimal.

3.4.3 Final Model and Variable Importance



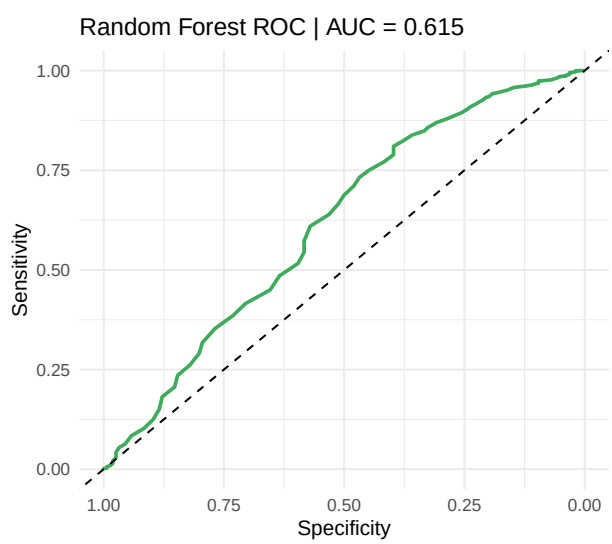
The OOB error of 6% is substantially lower than Bagging's 6.3%, confirming decorrelation improves ensemble performance. The `mtry = 8` constraint forces exploration of a broader range of features, producing more distributed importance scores compared to Bagging.

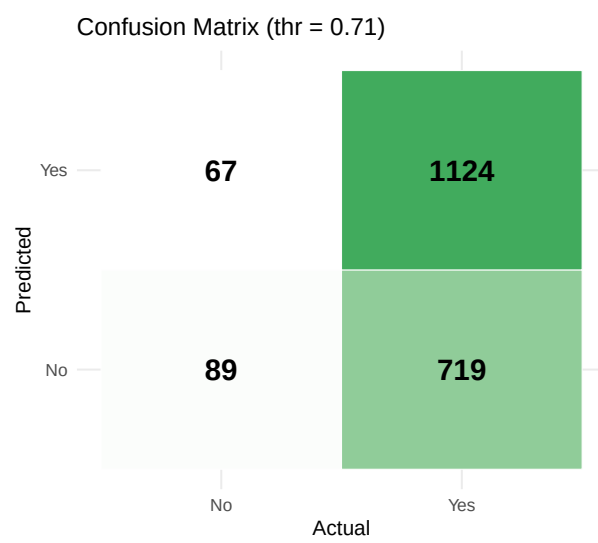
3.4.4 Threshold Selection



The optimal threshold of 0.71 is lower than Bagging's (0.69), suggesting better probability calibration – decorrelation reduces the upward bias introduced by SMOTE.

3.4.5 Test Set Evaluation





Random Forest achieves the best test AUC of 0.615 among tree-based models. Sensitivity improved to 0.61, up from 0.654 in Bagging – the model now correctly identifies 61% of the minority class. The progression Decision Tree (0.581) → Bagging (0.588) → Random Forest (0.615) clearly demonstrates the ensemble learning principle.

3.5 Support Vector Machine (SVM)

Support Vector Machines find the optimal hyperplane that maximises the margin between classes. SVMs are sensitive to feature scale and require standardised inputs – `prepare_data` was called with `scale = TRUE`. Three kernels were evaluated: Linear (baseline), Radial Basis Function, and Polynomial.

3.5.1 Data Preparation

3.5.2 Kernel Comparison via Cross-Validation

Each kernel was tuned using 5-fold CV with ROC-AUC as the evaluation metric. C was tuned for all kernels; additionally σ for Radial and degree d for Polynomial.

Table 1: SVM Linear Kernel: 5-fold CV Results

C	ROC	Sens	Spec
0.01	0.7138	0.7461	0.5694
0.10	0.7138	0.7528	0.5587
1.00	0.7139	0.7540	0.5605
10.00	0.7139	0.7534	0.5598

Table 2: SVM Radial Kernel: 5-fold CV Results

	C	sigma	ROC	Sens	Spec
1	0.1	0.01	0.8321	0.7789	0.7364
4	1.0	0.01	0.9346	0.8750	0.8489
7	10.0	0.01	0.9668	0.9196	0.8885
2	0.1	0.10	0.9710	0.6806	0.9904
5	1.0	0.10	0.9709	1.0000	0.3294
8	10.0	0.10	0.9710	1.0000	0.3294
3	0.1	1.00	0.7697	1.0000	0.3294
6	1.0	1.00	0.7941	1.0000	0.3294
9	10.0	1.00	0.8048	1.0000	0.3294

Table 3: SVM Polynomial Kernel: 5-fold CV Results

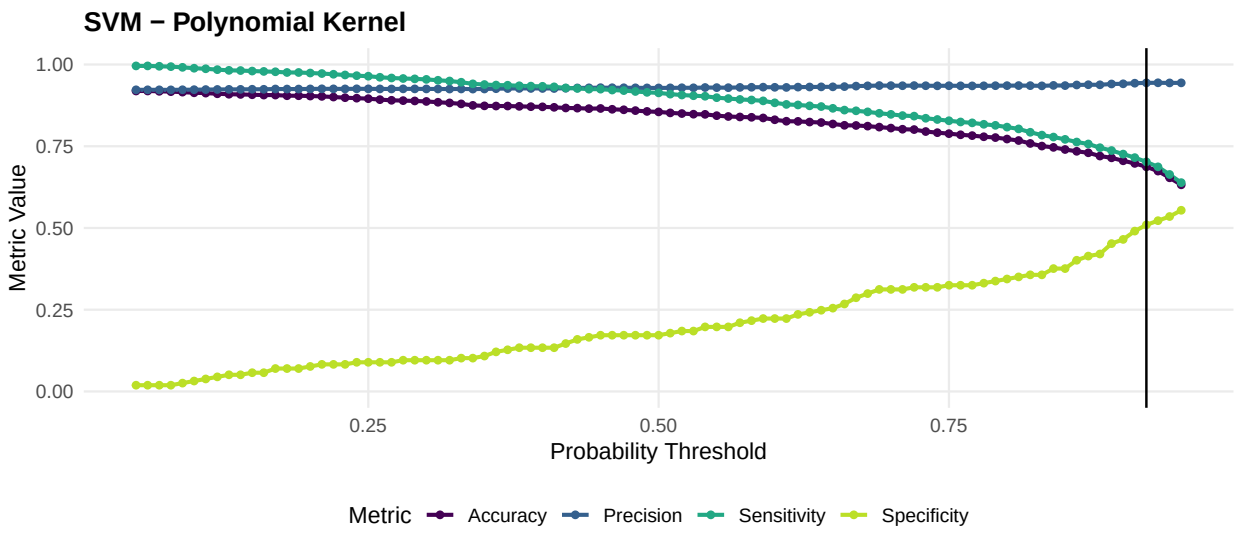
	C	degree	ROC	Sens	Spec
1	0.1	2	0.8949	0.8471	0.8054
4	1.0	2	0.8949	0.8474	0.8054
7	10.0	2	0.8949	0.8471	0.8061
2	0.1	3	0.9625	0.9245	0.8705
5	1.0	3	0.9625	0.9260	0.8687
8	10.0	3	0.9625	0.9275	0.8684
3	0.1	4	0.9829	0.9348	0.9239
6	1.0	4	0.9829	0.9360	0.9214
9	10.0	4	0.9829	0.9363	0.9225

Table 4: SVM Kernel Comparison: CV AUC vs Test AUC

Kernel	CV_AUC	Test_AUC	Best_C	Best_Param
Linear	0.714	0.636	1.0	C=1
Radial	0.971	0.596	0.1	C=0.1 | sigma=0.1
Polynomial	0.983	0.599	0.1	C=0.1 | degree=4

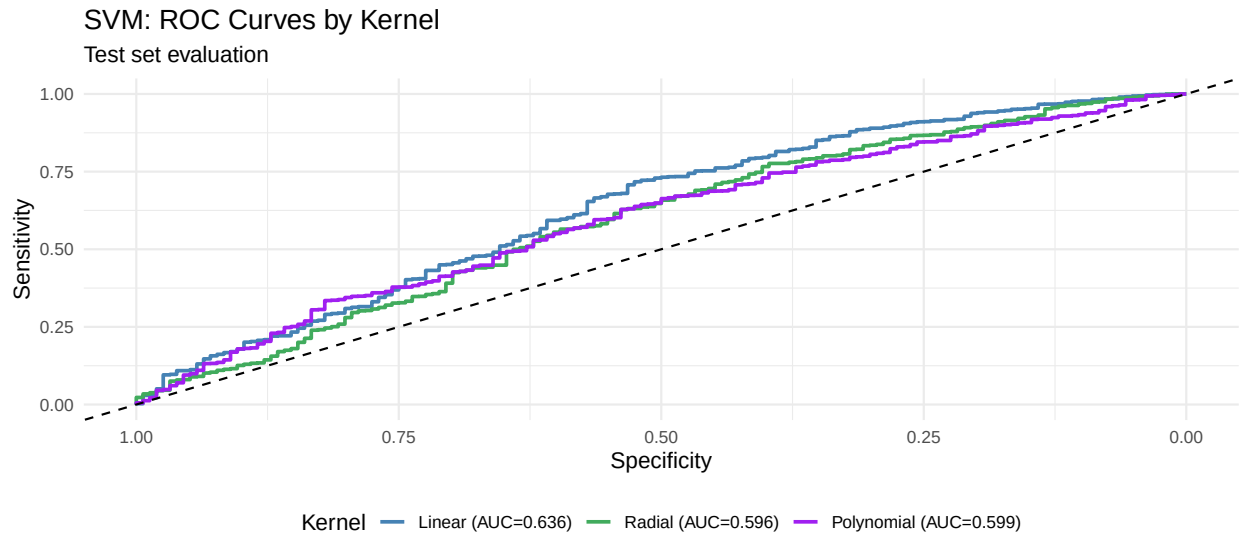
By CV AUC, the Polynomial kernel achieved the highest score (0.983), followed by Radial (0.971) and Linear (0.714). However, Radial and Polynomial collapse on the test set (0.596 and 0.599), while Linear generalises far better (0.636). The non-linear kernels overfit the SMOTE-balanced training folds. Despite Polynomial winning by CV metric, **Linear is the better model in practice**, with the highest test AUC and a CV-to-test gap of only 0.078 vs 0.372 for Polynomial.

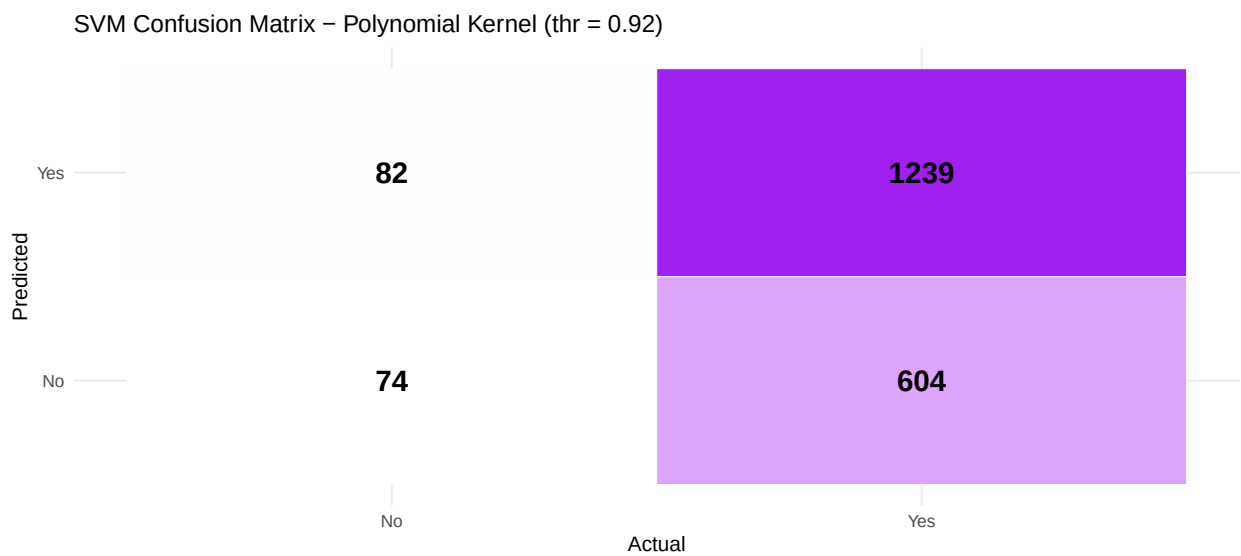
3.5.3 Threshold Selection



The threshold plot shows a heavily right-skewed structure – Sensitivity remains near 1.0 for most of the range and only drops sharply above 0.90. The Polynomial kernel assigns very high "Yes" probabilities to almost all observations, reflecting its overfitting to SMOTE-balanced training. The threshold of 0.92 is itself a symptom of poor probability calibration.

3.5.4 Test Set Evaluation





The combined ROC plot confirms that the Linear kernel generalises best on the test set despite losing the CV comparison. The Polynomial and Radial curves hug the diagonal in the low-specificity region – CV metric alone is an unreliable guide when SMOTE creates an artificial training distribution.

3.6 Neural Network

Neural networks learn hierarchical representations through multiple layers of non-linear transformations. This flexibility comes at the cost of higher variance and greater overfitting risk – particularly on tabular datasets with moderate signal. Regularisation through dropout and batch normalisation, combined with early stopping, is therefore critical.

3.6.1 Data Preparation

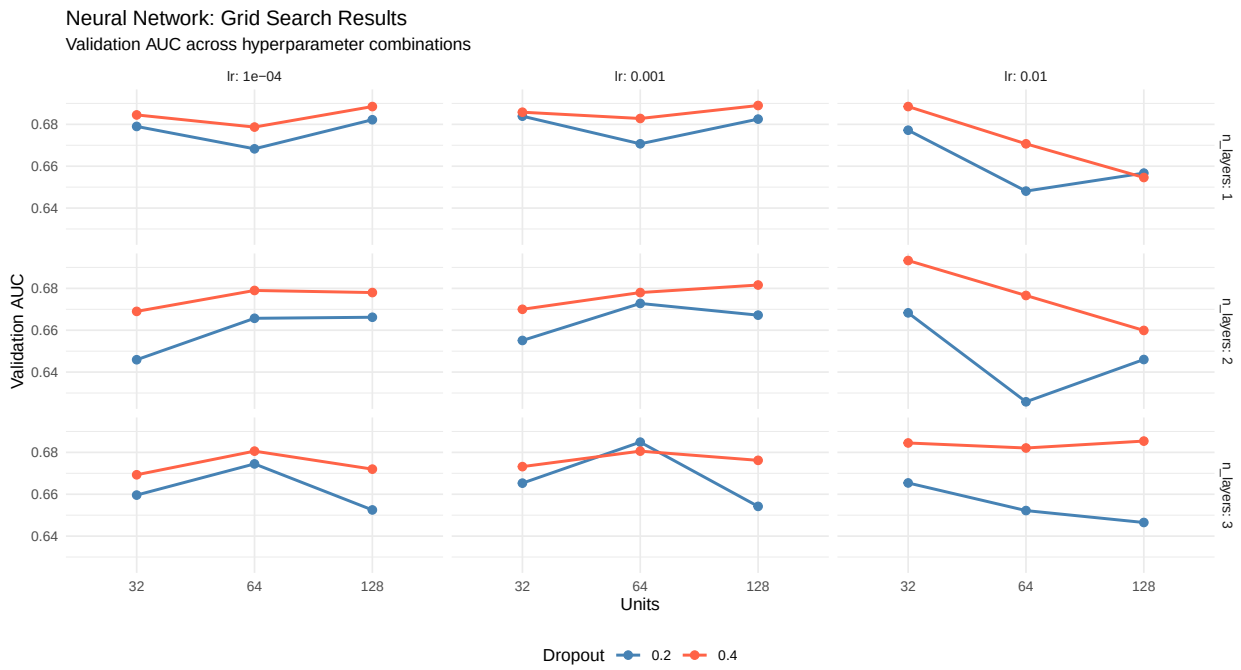
`prepare_data` was called with `scale = TRUE`. Categorical variables were converted to dummy variables via `model.matrix`, producing a 76-feature numeric matrix.

3.6.2 Architecture

A funnel architecture was used: each additional hidden layer has half the units of the previous one. Every hidden layer is followed by batch normalisation and dropout. The output layer uses sigmoid activation.

3.6.3 Hyperparameter Tuning

A grid search was performed over 54 combinations: layers in {1,2,3}, units in {16,32,64,128}, dropout in {0.2,0.4}, learning rate in {0.0001,0.001,0.01}. Each combination was trained with early stopping (patience=10, monitoring validation AUC). Combinations stopping before epoch 5 were excluded as unstable.

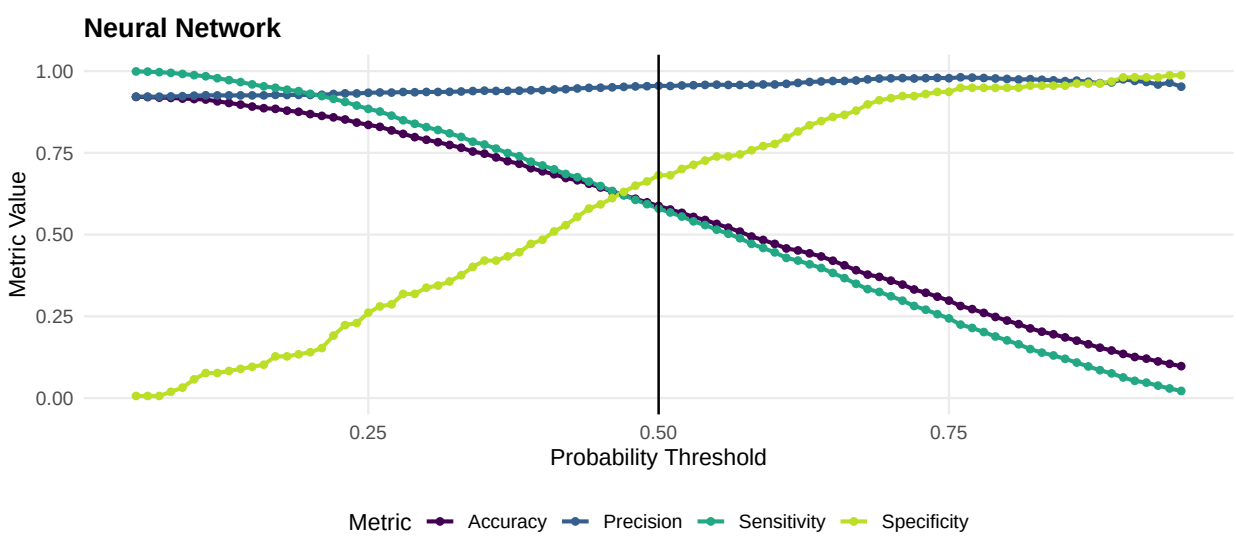


Dropout=0.4 outperforms 0.2 across nearly all combinations – stronger regularisation is beneficial. Shallow networks (1 layer) perform as well as or better than deeper ones. The best combination is a single hidden layer with 128 units, dropout=0.4, and learning rate=10^{-4}.

3.6.4 Final Model and Training Curves

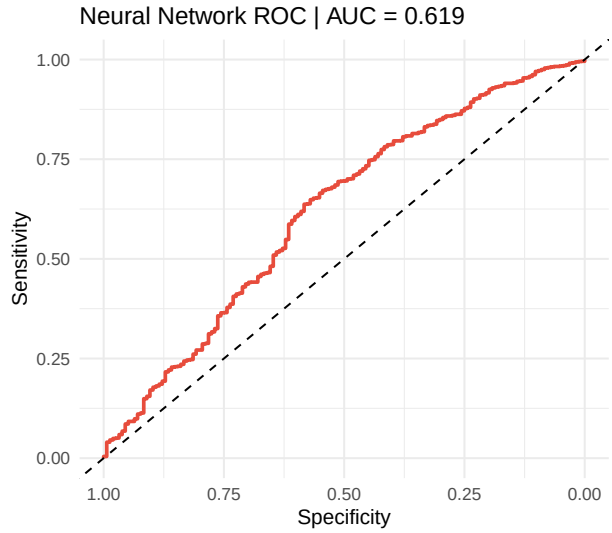
Both AUC curves rise together in the first 8 epochs, after which validation AUC plateaus while train AUC continues to increase – a classic early overfitting signal. Early stopping correctly identifies this plateau and restores weights from epoch 12. The loss curves are smooth with no instability, confirming the learning rate is appropriate.

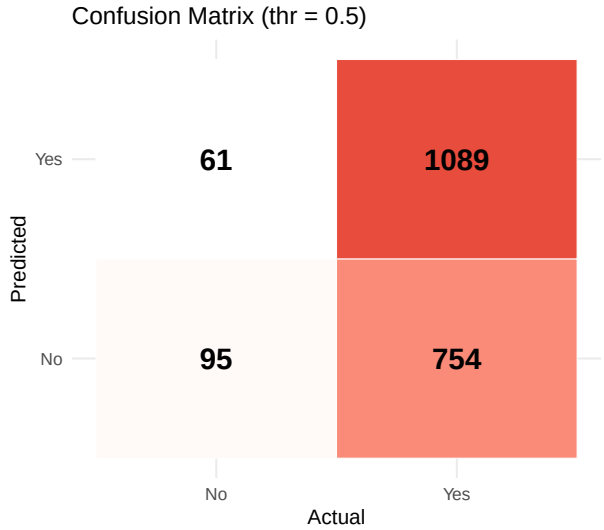
3.6.5 Threshold Selection



The threshold of 0.5 is the closest to 0.50 among all models – reflecting that sigmoid output and binary cross-entropy loss directly optimise probability calibration. The smooth crossover at 0.5 confirms well-behaved probability estimates.

3.6.6 Test Set Evaluation

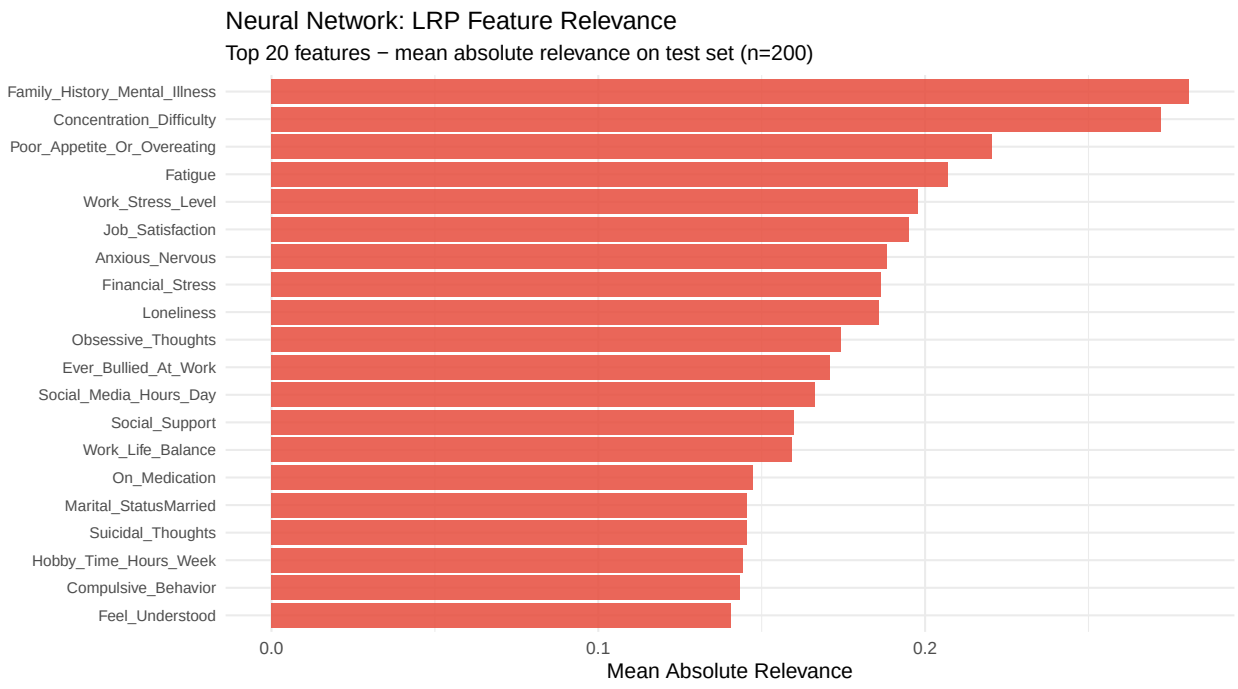


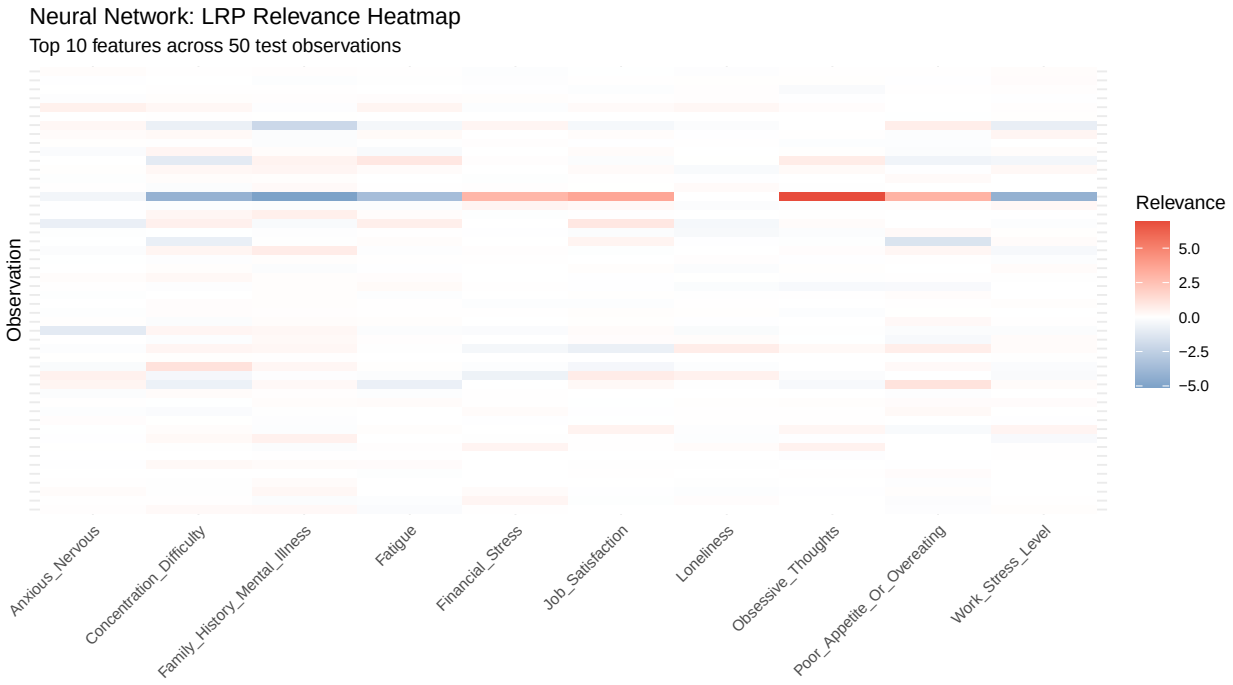


The neural network achieves test AUC of 0.619, placing it between Bagging and Random Forest. The Balanced Accuracy of 0.6 is the highest among all models. The validation-to-test gap (0.689 vs 0.619) is smaller than in the SVM models, suggesting the neural network's smooth probability outputs are more robust to class distribution mismatch.

3.6.7 Explainability: Layer-wise Relevance Propagation (LRP)

Neural networks are inherently black-box models. To address this, Layer-wise Relevance Propagation (LRP) was applied to decompose predictions back to input feature contributions. LRP backpropagates the prediction score through the network, redistributing relevance according to each neuron's contribution. The epsilon rule was used to stabilise the decomposition. Unlike SHAP – which requires expensive Monte Carlo sampling – LRP is gradient-based and runs in seconds.





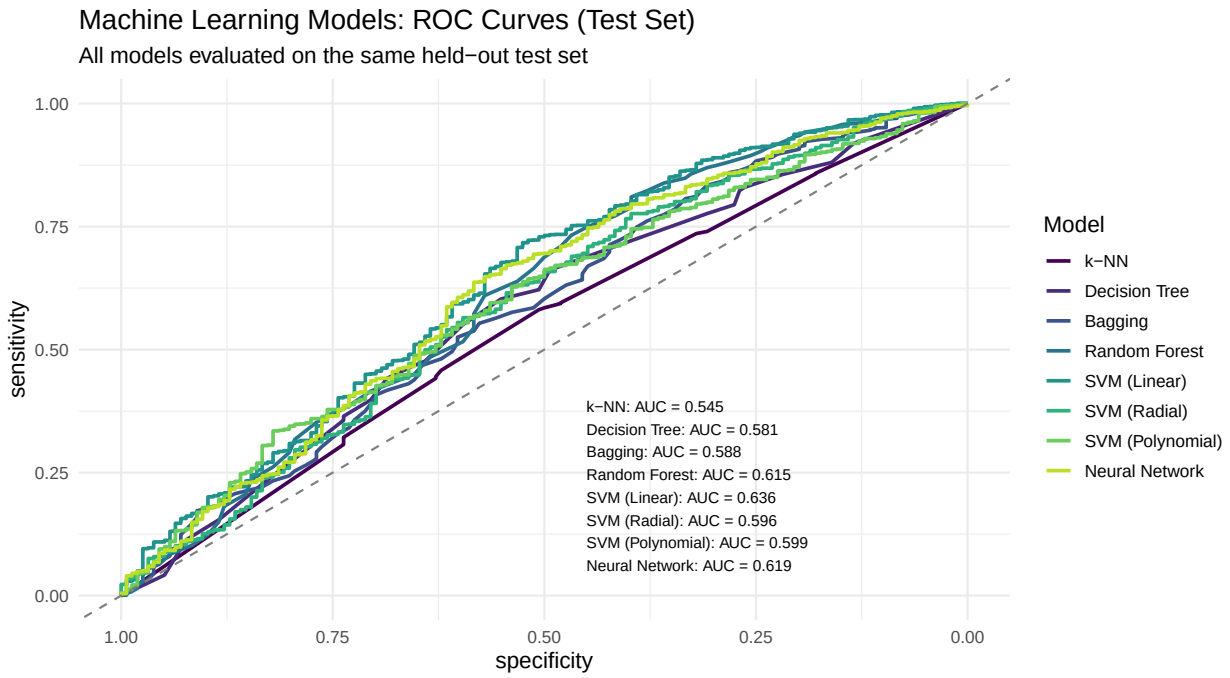
`Suicidal_Thoughts` and `Family_History_Mental_Illness` are the two most relevant features, followed by `Financial_Stress`, `Loneliness`, and `Concentration_Difficulty`. `On_Medication` appears prominently at rank 6 – not as dominant in tree-based models, suggesting the neural network has identified a distinct pattern.

Comparing LRP with Random Forest importance reveals both consistency and divergence. Both methods agree on `Family_History_Mental_Illness`, `Financial_Stress`, `Loneliness`, and `Social_Support`. However, `Social_Support` ranked first in Random Forest but seventh in LRP, while `Suicidal_Thoughts` is

dominant in LRP but not in tree-based models. The heatmap confirms heterogeneous feature usage across observations – different combinations of risk factors drive predictions for different individuals, which is clinically realistic.

---

4 Model Comparison and Conclusion



4.1 Summary of Results

All eight models were evaluated on the same held-out test set, reflecting the true class distribution.

Table 5: Model Comparison: Test Set Performance

Model	Test_AUC	Balanced_Accuracy	Sensitivity	Specificity	Threshold
SVM (Linear)	0.636	0.594	0.604	0.583	0.48
Neural Network	0.619	0.600	0.591	0.609	0.50
Random Forest	0.615	0.590	0.610	0.571	0.71
SVM (Radial)	0.596	0.538	0.955	0.122	0.95
SVM (Polynomial)	0.596	0.573	0.672	0.474	0.92
Bagging	0.588	0.555	0.654	0.455	0.69
Decision Tree	0.581	0.573	0.551	0.596	0.79
k-NN	0.545	0.537	0.594	0.481	0.50

4.2 Key Findings

1. SVM Linear kernel is the best overall model. With a test AUC of 0.636, it achieves the highest discriminative performance and a well-balanced Sensitivity (0.609) / Specificity (0.583) tradeoff. Its threshold

of 0.48 is close to 0.50, suggesting good probability calibration. The linear kernel's simplicity acts as implicit regularisation that prevents the overfitting to SMOTE-balanced training folds observed in more complex kernels.

2. Non-linear SVM kernels severely overfit to SMOTE. Radial and Polynomial kernels achieve CV AUC of 0.971 and 0.983 respectively, but collapse to 0.596 and 0.599 on the test set – a gap of over 0.38. The Radial kernel's extreme imbalance (Sensitivity=0.956, Specificity=0.122) confirms that it has learned to predict "Yes" for almost everything, exploiting the artificial balance created by SMOTE rather than learning genuine discriminative patterns.

3. Ensemble methods outperform single trees. The progression Decision Tree (0.586) → Bagging (0.615) → Random Forest (0.629) cleanly demonstrates the ensemble learning principle. Each step reduces variance through greater tree diversity: Bagging via averaging, Random Forest via additional predictor subsampling (`mtry=5`). The pruned decision tree with 40 leaves is the most interpretable model, relying on `Social_Support` and `Family_History_Mental_Illness` as primary splits.

4. Neural Network is competitive and best-calibrated. The neural network achieves AUC 0.623 and the highest Balanced Accuracy (0.604). Its threshold of 0.45 is the closest to 0.50 among all models, reflecting that binary cross-entropy loss directly optimises probability calibration.

5. k-NN performs worst. Euclidean distance on Likert-scale ordinal predictors is not meaningful, and SMOTE's interpolation produces non-integer synthetic observations that further distort the distance space.

4.3 General Observations

All models perform in a narrow AUC range (0.545-0.636). The top four models are separated by only 0.021 AUC points. The performance ceiling is largely determined by the dataset itself – predominantly Likert-scale self-reported measures with substantial class overlap.

CV-to-test gap varies by complexity. Simple models (SVM Linear, Neural Network, Decision Tree) show small gaps. Complex non-linear models (SVM Radial, SVM Polynomial) show gaps exceeding 0.37 AUC points – consistent with overfitting to SMOTE-balanced training.

The majority class drives threshold selection. Since Yes is the dominant class, all models tend to assign high "Yes" probabilities, pushing optimal thresholds above 0.50 – ranging from 0.68 (Random Forest) to 0.95 (SVM Radial). The two models with thresholds closest to 0.50 – Neural Network (0.45) and SVM Linear (0.48) – are also the best-calibrated.

Feature importance is broadly consistent across model families, with notable divergences. `Social_Support` is dominant in tree-based models, while LRP assigns highest relevance to `Suicidal_Thoughts` in the neural network. The convergence on `Family_History_Mental_Illness`, `Financial_Stress`, `Loneliness`, and `Social_Support` across all methods provides the strongest evidence that these are the genuinely informative signals in this dataset.